Ders 12 Genişletilmiş Notu: Quiz ve Ödev Çözümleri: Renk Histogramı, Dinamik Bellek ve Yığınla Kelime Çözme

Ders 12 Genişletilmiş Notu: Quiz ve Ödev Çözümleri: Renk Histogramı, Dinamik Bellek ve Yığınla Kelime Çözme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Oturumun Konusu ve İki Soru
- 2. Quiz 1: Renk Matrisinin Histogramı
- 3. malloc, calloc ve Çözüm Kodu
- 4. Çıktıyı Hizalı Yazdırmak
- 5. Belleği Daha İyi Kullanmak: realloc ve Bağlı Liste
- 6. Sabit Boyut mu, Dinamik Bellek mi?
- 7. Kod Yazma Alışkanlıkları ve Global Değişkenler
- 8. Ödev 1: Yığınla Kelime Çözme
- 9. İki Kelimeyi Yığınla Karşılaştırmak
- 10. Hazır Fonksiyon Ne Zaman Kullanılır?

1. Oturumun Konusu ve İki Soru

Bu kayıt bir uygulama oturumudur. Dersi bir araştırma görevlisi anlattı ve daha önce verilen iki sorunun çözüm yaklaşımını gösterdi. Bu nottaki “Hocanın vurgusu” blokları da araştırma görevlisinin vurgularıdır.

İki soru vardı:

- **Quiz 1:** İki boyutlu bir renk matrisinin histogramı çıkarılacaktı. Bazı varsayımlar verilmişti. Bu varsayımlara göre bir algoritma geliştirip kodunu yazmak gerekiyordu.
- **Ödev 1:** Bir yığın (stack) sorusuydu. İki kelime veriliyor, program karakterleri kontrol edip kelimelerin eşit olup olmadığını söylüyordu.

Quiz 1’in kodu ekranda gösterildi. Ödev 1’in ise yalnız yöntemi anlatıldı, kodu gösterilmedi. Araştırma görevlisi kodu göstermekteki amacının eksikleri görmek ve fikir vermek olduğunu söyledi. Veri yapıları dersinde dizi, bağlı liste gibi yapıların önemi çok büyüktür, çünkü dersin amacı bu yapıları ve uygun kullanımlarını vermektir.

> **Hocanın vurgusu:** Veri yapıları dersinde asıl mesele doğru yapıyı seçmek ve onu uygun kullanmaktır.

2. Quiz 1: Renk Matrisinin Histogramı

Elimizde $N \times M$ boyutlu bir renk matrisi vardır. Renk değerleri 0 ile 255 arasındadır. Amaç, matristeki farklı renkleri ve her birinin kaç kez geçtiğini bulmaktır.

N ve M değerleri kodda makro ile tanımlandı. Aynı değer kodun birçok yerinde kullanıldığı için makro yönetimi kolaylaştırır. Araştırma görevlisi test matrisini de hızlıca ekrana girdiğini söyledi.

Soruda farklı renk sayısı için bir varsayım vardı. Soru metnindeki $k/3$ değişkenine araştırma görevlisi kısalık için kodda `k` dedi. En büyük renk aralığı 256 olduğundan, tamsayı bölmesiyle (ara adım eklendi):

$$k = \lfloor 256/3 \rfloor = 85$$

2.1 Algoritmanın Mantığı

Çözümde indisleri ortak iki dizi kullanılır:

- `uniques`: Bulunan farklı renkleri tutar.
- `counts`: Her rengin kaç kez geçtiğini tutar. `counts[t]`, `uniques[t]` renginin sayısıdır.

Bir de `found` bayrağı (flag) vardır. Bu bayrak, sıradaki pikselin rengi `uniques` dizisinde bulundu mu sorusunu cevaplar. `colorCount` ise şimdiye kadar kaç farklı renk görüldüğünü sayar.

Algoritmanın iki katmanı vardır. Dış katman bütün pikselleri gezer. Bunun maliyeti $O(N \cdot M)$'dir.

> **Hocanın vurgusu:** Bütün pikselleri gezmek zorundayız. Bu adımın karmaşıklığı $N \cdot M$'dir ve bundan kaçış yoktur.

İç katman ise kısaltılabilir. Bir pikselin rengi `uniques` dizisinde bulunduğu anda aramayı durdurabiliriz, dizinin sonuna kadar gitmek gerekmez. Bu yüzden iç döngü `for` değil, iki koşullu bir `while` döngüsüdür:

- Renk bulunursa `found` 1 olur ve çıkış koşulu sağlanır. O rengin sayacı da bir artırılır.
- Renk bulunamazsa `t`, `colorCount` değerine ulaşınca döngü biter.

Döngüden çıkıldığında bu iki durumdan biri olmuştur. Hangisi olduğu değişkenlere bakarak anlaşılır. Renk bulunamadıysa bu yeni bir renktir. `colorCount` numaralı göze yazılır, sayacı 1'den başlar ve `colorCount` bir artar.

> **Ek bilgi (derste anlatılmadı):** Derste yalnız piksel gezme adımının maliyeti söylendi. İç `while` en çok `colorCount` kez döndüğü için toplam maliyet en kötü durumda $O(N \cdot M \cdot k)$ olur. C farklı renk varsa karşılaştırma sayısı en çok $\frac{C(C-1)}{2} + (NM - C)C$'dir. Bu üst sınır python ile rastgele girdilerde sınandı, aşılmadı ve en kötü dizilimde tam olarak ulaşıldı.

2.2 İki Dizi Yerine Tek Matris

`uniques` ve `counts` tek bir yapıda da tutulabilirdi. Bu, T satırlı ve iki sütunlu tek bir matris olurdu. 0. sütunda renkler, 1. sütunda sayılar dururdu. Araştırma görevlisi iki ayrı dizi kullanmayı tercih etti, ama iki yol da mümkündür.

3. malloc, calloc ve Çözüm Kodu

Bu çözümde önceden yer ayırma (pre-allocation) yapıldı. Koda iki farklı ayırma biçimi örnek olsun diye birlikte kondu:

- **`malloc`:** Tek parametre alır. Bu parametre ayrılacak alanın boyutudur.
- **`calloc`:** İki parametre alır: eleman sayısı ve her elemanın boyutu. Ayrıca ayırdığı alanı sıfırla doldurur.

> **Ders sonrası not:** Araştırma görevlisi `malloc` parametresinin bit sayısı olduğunu söyledi; doğrusu bayt sayısıdır. Koddaki `k * sizeof(int)` ifadesi de bayt verir, çünkü `sizeof` sonucu bayt cinsindendir. Araştırma görevlisi `calloc` parametrelerini önce boyut, sonra sayı diye saydı; doğru sıra koddaki gibi `calloc(k, sizeof(int))`, yani önce sayı, sonra boyuttur.

Çözüm kodunun ana kısmı şöyledir:

```
int maxColorRange = 256; // 0-255

int i, j, t; // indisler
int k = (int)maxColorRange / 3; // varsayima gore kabul edilen "k/3" farkli renk sayisi
int found; // histogram icin kullanilan flag.
int colorCount;

int* uniques = (int*)malloc(k * sizeof(int));
int* counts = (int*)calloc(k, sizeof(int));

colorCount = 0;
for (i = 0; i < N; i++)
{
    for (j = 0; j < M; j++)
    {
        found = 0;
        t = 0;
        while ((found == 0) && t < colorCount) {
            if (uniques[t] == test[i][j]) {
                found = 1;
                counts[t] = counts[t] + 1;
            }
            t++;
        }
        if (found != 1) {
            uniques[colorCount] = test[i][j];
            counts[colorCount] = 1;
            colorCount++;
        }
    }
}
```

> **Ders sonrası not:** Bu parça kısa nottaki ekran aktarımından alındı. `N`, `M` ve `test` kodun bu parçada görünmeyen kısmında tanımlıdır. Parça, bu üç tanım ve `free` çağrıları eklenerek `gcc -Wall -Wextra` ile uyarısız derlendi, çalıştırıldı ve bellek denetleyicisinde (AddressSanitizer) hata vermedi. Kod `colorCount < k` denetimi yapmaz. Bu yüzden yalnız soru varsayımı, yani en çok k farklı renk olması sağlanırsa güvenlidir. Renk sayısı k'yi aşan bir girdide AddressSanitizer taşma bildirdi.

4. Çıktıyı Hizalı Yazdırmak

Sonuçlar iki `for` ile ekrana yazdırıldı. Araştırma görevlisi burada bir biçimlendirme ayrıntısına dikkat çekti. Renkler 0-255 aralığında olduğu için en çok üç basamaklıdır. Sayılar `%d` ile yazılırsa 1 ile 255 aynı hizadan başlar ve alt alta düzgün durmaz. `%3d` ile yazılırsa her sayı üç karakterlik alana yerleşir. Örneğin 1 sayısı `"  1"`, 255 sayısı `"255"` olarak yazılır. Bu çıktılar derlenen kodda doğrulandı. Sonuç göze daha hoş gelir ve daha kolay okunur.

5. Belleği Daha İyi Kullanmak: realloc ve Bağlı Liste

Soruda gömülü sistem (embedded system) senaryosu vardı, yani bellek kullanımı önemliydi. Yine de k varsayımı renk sayısını zaten kısıtladığı için buna çok takılmak gerekmiyordu. Araştırma görevlisi gösterilen çözümün belleği çok verimli kullandığını söyleyemeyeceğini belirtti. Baştan k kadar yer ayrılır. Renk sayısı azsa bu alanın bir kısmı boş kalır. Gömülü sistem için daha iyi iki tasarım anlatıldı.

5.1 Birinci Yol: realloc

`realloc`, daha önce ayrılmış bir bellek alanını genişletmek için kullanılır. Eski işaretçiyi ve yeni boyutu alır. Örneğin `counts` dizisi k elemanlık ayrılmışsa, daha büyük bir değer verilerek aynı işaretçi için alan büyütülür:

```
counts = (int*)realloc(counts, yeniK * sizeof(int));
```

Quiz çözümünde önceden ayrılmış dizinin gözleri tek tek doldurulur. Bunun yerine her yeni renk geldiğinde dizi büyütülürse bellek daha verimli kullanılır.

Bunun bir bedeli vardır. Ayırma işlemleri de birer fonksiyondur ve arka planda iş yaparlar. Sistem çağrıları yaparlar, bellekteki parçaları (fragment) birleştirip gereken alanı oluşturmaya çalışırlar. Yani `realloc` bellek kazandırır, ama CPU ve zaman harcatır. Gömülü sistemde bu da değerlendirilmesi gereken bir ölçüttür. Araştırma görevlisi kendi çözümünde önceden belirlenen k değerine bağlı kaldı ve bunun bir tercih olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** `realloc` alanı başka bir yere taşıyabilir, bu yüzden dönen adres kullanılmalıdır. Başarısız olursa `NULL` döner ve eski alan serbest kalmaz. `p = realloc(p, ...)` yazımında bu durumda eski adres kaybolur. Bu nedenle sonuç önce geçici bir işaretçiye alınıp denetlenebilir.

5.2 İkinci Yol: Bağlı Liste

`uniques` ve `counts` dizileri yerine bağlı liste (linked list) kullanılabilirdi. Dizide bellek ayrılmıştır ve `realloc` yapılmadıkça kullanılsa da kullanılmasa da öyle durur. Bağlı listede ise her yeni renk için yeni bir düğüm oluşturulur ve adresi listenin son düğümüne bağlanır. Araştırma görevlisine göre böylece bellek daha verimli kullanılır ve yönetim `realloc`'tan kolaydır.

Listenin iki dezavantajı vardır:

- **Ek adres:** Her düğüm bir sonraki düğümün adresini tutar. Eleman başına daha çok yer gerekir. Araştırma görevlisi bir `sizeof(int)` yerine belki iki `sizeof(int)` gerekeceğini söyledi. Adresin boyutu da sisteme bağlıdır.
- **Rastgele erişim (random access) yok:** Dizide `uniques[5]` diyerek o göze doğrudan gidilir ve kaç kez geçtiğine bakılır. Listede doğrudan adresleme yoktur. Beşinci göze ulaşmak için baştan başlanıp ikinci, üçüncü, dördüncü ve beşinci eleman tek tek gezilir.

Araştırma görevlisi uygulama gömülü sistem için tasarlandığından, bu dezavantajlara rağmen bellek ayırma açısından bağlı listenin daha avantajlı sayılabileceğini söyledi.

> **Ders sonrası not:** Araştırma görevlisi adres için `int` yerine `long int` gibi bir tamsayı tanımlamak gerekebileceğini söyledi; C'de adres işaretçi (pointer) tipinde tutulur. Derlenen kodda (x86-64, gcc) `int` 4, işaretçi 8 bayt çıktı. `deger`, `sayi` ve `next` alanlı bir düğüm 16 bayt çıktı. Dersteki iki dizi $2 \times 85 \times 4 = 680$ bayt tutar. Liste yalnız düğüm boyutuyla sayılırsa $680/16 = 42{,}5$ olduğundan en çok 42 renkte diziden az yer kaplar, yani kazanç renk sayısına bağlıdır. Liste de her düğüm için `malloc` çağırır. Ayırıcının işi kalkmaz.

5.3 Listede Yapı Tanımı ve head

Soru: Bağlı listenin başı (`head`) global olabilir mi?

Cevap: `head` global olmak zorunda değildir. Sonuçta bir işaretçidir ve `main` içinde tutulabilir. Yapı (structure) tanımı ise farklıdır. O bir değişken değil, bir tanımdır. Araştırma görevlisi yapıyı, kullanılacak değişkenleri tutan bir kapsüle benzetti. Bu kapsül global tanımlanabilir. `main` içinde ondan bir işaretçi tanımlandığında artık bir değişken olur. Bu işaretçi listenin başını tuttuğu için `main` içinde olması daha doğrudur.

> **Ders sonrası not:** Araştırma görevlisi yapı tanımının global olmak zorunda olduğunu söyledi; bu zorunlu değildir. Fonksiyon içinde yapı tanımı da derlenir, ama yalnız o blokta görünür. Yapıyı birden fazla fonksiyon kullanacaksa dosya başında tanımlamak gerekir.

Ek bilgi (derste anlatılmadı): Derste liste kodu gösterilmedi. Aşağıdaki iskelet, yapı tanımının dosya başında, `head` işaretçisinin `main` içinde durmasını somutlaştırır. `head` fonksiyona parametre olarak gider. Kod `gcc -Wall` ile uyarısız derlendi ve çalıştırıldı.

```c
#include <stdio.h>

struct renk {
    int deger;
    int sayi;
    struct renk *next;
};

void yazdir(struct renk *head) {
    struct renk *p;
    for (p = head; p != NULL; p = p->next)
        printf("%3d %3d\n", p->deger, p->sayi);
}

int main(void) {
    struct renk *head = NULL;
    yazdir(head);
    return 0;
}
```

6. Sabit Boyut mu, Dinamik Bellek mi?

Soru: `uniques` ve `counts` dizilerini sabit boyutlu oluştursaydık aynı şey olmaz mıydı?

Cevap: Bu kodda doğrudur, sabit boyut verilerek de yapılabilirdi. `malloc` ve `calloc` burada ayırma yöntemlerinden söz etmek için kullanıldı. Ama gerçek programda en büyük renk değeri gibi girdiler kullanıcıdan okunmalıdır. Araştırma görevlisi `maxColorRange` değerinin ekrandan okutulması gerektiğini hatırladığını söyledi. Bu değer okunmadan k bilinmez ve kaç eleman ayrılacağı önceden belli olmaz. O durumda `malloc` veya `calloc` kullanmak zorunludur.

> **Hocanın vurgusu:** Ayırmaları mümkün olduğunca dinamik yapmaya alışın. N, M gibi değerleri sabit vermek yalnız kendi kodunuzu denerken uygundur.

Dinamik ayırmanın ikinci yarısı belleği geri vermektir. C ve C++'ta Java ve Python'daki gibi çöp toplayıcı (garbage collector) olmayabilir. Ayrılan her alan, kullanımı bitince `free` ile belleğe geri verilmelidir. Bu kodda iki `free` çağrısı gerekir:

```c
free(uniques);
free(counts);
```

> **Hocanın vurgusu:** Ayrılan belleğin kullanım bitince `free` ile geri verildiğini görmek istiyoruz.

7. Kod Yazma Alışkanlıkları ve Global Değişkenler

7.1 Anlamlı Değişken Adları

Değişken adı ne tuttuğunu anlatmalıdır. `maxColorRange` iyi bir örnektir. Rengin alabileceği en büyük aralığı açıklar. Yanındaki `// 0-255` yorumu da renk skalasını gösterir. Buna karşılık `int a = 256;` kötü bir tanımdır. Ne olduğunu anlamak için kodda kullanıldığı yere gitmek gerekir.

İndisleri ayrı bir yerde, gruplayarak tanımlamak da yorum yazmayı ve kodu okumayı kolaylaştırır.

7.2 Global Değişkenler Neden Sakıncalı?

> **Hocanın vurgusu:** Global değişken görmek istemiyoruz.

Araştırma görevlisi konuyu üstünkörü anlattığını, ayrıntısının işletim sistemleri dersinde görüleceğini söyledi. Temel sorun şudur: Birden fazla iş parçacığı (thread) ile çalışırken aynı global değişkene erişen iki iş parçacığı yarış durumuna (race condition) düşebilir. Değişken, olması gerekenden çok farklı bir değer tutabilir.

Ekranda sözdizimine dikkat edilmeden şöyle bir örnek yazıldı:

```
int globalVal = 0;

X iş parçacığı: önce başka işler yapar, sonra globalVal = 1 yapar
Y iş parçacığı: globalVal = 0 yapar
```

Programcı önce X'in, sonra Y'nin çalışmasını bekler. Bu sırada değişkenin son değeri 0 olur. Sıra tersine dönerse son değer 1 olur (ara adım eklendi). Sıra ise programcının elinde değildir. Kesintili zamanlayıcı (preemptive scheduler) bir X'i, bir Y'yi çalıştırır, belki Y'yi üst üste iki üç kez çalıştırır. İş parçacıkları arasındaki kesintiler de kontrol edilemez. Sonuçta kaotik sonuçlar alınabilir. Bu tür bir hata genelde fark edilmez ve çözümlemesi çok zordur.

> **Ders sonrası not:** Altyazıda zamanlayıcı işlemciyle birlikte anılıyor ve iş parçacıklarının değişkenin adresini değiştirdiği söyleniyor. Zamanlayıcı işletim sisteminin bir parçasıdır. İş parçacıkları değişkenin adresini değil değerini değiştirir.
>
> **Ek bilgi (derste anlatılmadı):** İşletim sistemleri dersine bırakılan kritik bölge (critical section), paylaşılan veriye erişen kod parçasıdır. Mutex ise bu bölgeye aynı anda tek iş parçacığının girmesini sağlayan kilittir.

Global değişkene karşı ikinci neden, onunla yapılan her şeyin yerel değişkenle de yapılabilmesidir. Global tanımlamanın gerekçesi genelde değişkeni birden fazla fonksiyonda kullanmaktır. Oysa değer fonksiyona parametre olarak gönderilebilir. Örneğin `maxColorRange`, k hesaplanırken kullanılıyor. Bu hesap bir fonksiyonda olsa, değeri yukarıda global tanımlamak yerine parametre olarak göndermek yeterlidir.

> **Hocanın vurgusu:** Global değişken kullanmak bir zorunluluk değil, bir tercihtir. Bu tercihe alışmanızı istemiyoruz.

8. Ödev 1: Yığınla Kelime Çözme

Ödevde iki kelime veriliyor ve eşit olup olmadıkları soruluyordu. Soru basit olduğu için birçok farklı çözüme açıktır. Soruda özellikle yığın kullanın denmedi, başka çözümler de geçerlidir. Beklenen ise bunun bir yığın sorusu olduğunu görmekti.

Örnek şudur: `"Veri"` ile `"Vezir3 Rize2"` eşit mi? İkinci yazım kodlanmış (encoded) bir kelimedir. Rakam, kendisinden önceki son kaç harfin silineceğini söyler. `Vezir3` için son üç harf (`zir`) silinir, geriye `Ve` kalır. `Rize2` için son iki harf (`ze`) silinir, geriye `Ri` kalır. Birleşince `Veri` elde edilir.

> **Ders sonrası not:** Altyazıda bu adımda geriye "V" ve "r" kaldığı söyleniyor. Bu büyük olasılıkla altyazı bozulması ya da söyleyiş kısalığıdır. Doğru kalan kısımlar `Ve` ve `Ri`'dir. Aşağıdaki iz C ve python ile doğrulandı.

8.1 Bunun Yığın Sorusu Olduğu Nasıl Anlaşılır?

İpucu okuma ve silme yönlerinin farkıdır. Kelimeyi soldan sağa okuruz. Üç harf silineceği zaman ise sondan başa doğru gideriz. En son gelen harf ilk silinir. Bu, itme (push) ve çekme (pop) mekanizmasıdır.

Yöntem şöyledir:

- Rakam görene kadar her harf yığına itilir.
- Rakam gelince o sayı kadar çekme yapılır.
- Girdi bitince yığında çözülmüş kelime kalır.

8.2 Adım Adım İz

Girdi	İşlem	Çekilen harfler	Yığın (alttan üste)
`V e z i r`	5 push		`V e z i r`
`3`	3 pop	`r, i, z`	`V e`
`R i z e`	4 push		`V e R i z e`
`2`	2 pop	`e, z`	`V e R i`

Bu, programın birinci aşamasıdır. Kodlanmış hâl normal bir karakter dizisine çevrilmiş, yani çözülmüş (decode) olur. İkinci aşamada çözülen kelime orijinal kelimeyle karşılaştırılır.

> **Ders sonrası not:** Araştırma görevlisi sonucu `Veri` ile "aynı kelime" saydı, yani örnekte büyük/küçük harf ayrımı yapmadı. Kısa nottaki ekran aktarımına göre yığında `VeRi` kalır. C'de `'R' == 'r'` yanlıştır. Bu yüzden harf duyarlı karşılaştırma bu girdide "eşit değil" sonucu verir. Derlenen kodda harf duyarlı karşılaştırma eşit değil, harf duyarsız karşılaştırma eşit sonucunu verdi. Kodlanmış yazımdaki boşluk da derlenen sınamada atlandı. Ödev metnindeki kural altyazıda yoktur.

9. İki Kelimeyi Yığınla Karşılaştırmak

C'de iki yol vardır. Birincisi hazır `strcmp` fonksiyonudur. Araştırma görevlisi bunu kullanmak yerine yığın yaklaşımını tavsiye etti.

> **Ders sonrası not:** Araştırma görevlisi `strcmp`'nin iki ya da üç parametre alabildiğini söyledi; `strcmp` iki parametre alır. Üç parametreyle çağrı `gcc` ile derleme hatası verdi. İlk n karakteri karşılaştıran üç parametreli fonksiyon `strncmp`'dir. Parametreler de "birinci ve ikinci karakter" değil, iki karakter dizisinin adresidir.

Yığın yaklaşımı şöyledir. Orijinal kelime de aynı biçimde ayrı bir yığına itilir. Sonra iki yığından aynı anda birer karakter çekilir ve karşılaştırılır. C'de tek karakter `if` içinde `==` ile doğrudan karşılaştırılabilir, ama string bu şekilde karşılaştırılamaz.

> **Ders sonrası not:** İki karakter dizisi `==` ile yazılırsa derleme yine de geçebilir. `gcc -Wall` uyarı verdi (`comparison between two arrays`). İçerikleri aynı olan iki dizi için sonuç 0 çıktı, çünkü karşılaştırılan şey içerik değil adrestir.

Tahtadaki kelimede karşılaştırma üstten alta şöyle ilerler (harf farkı yok sayılarak, ara adım eklendi):

Adım	Çözülen yığından pop	Orijinal yığından pop	Eşit mi?
1	`i`	`i`	Evet
2	`R`	`r`	Evet (harf farkı yok sayılırsa)
3	`e`	`e`	Evet
4	`V`	`V`	Evet
5	Yığın boş	Yığın boş	Kelimeler eşit

Bütün karakterler eşitse ve iki yığın aynı anda boşalırsa kelimeler eşittir. Araştırma görevlisi iki yığının boyunun eşit olacağını da belirtti. Yığınlardan biri ötekinden önce boşalırsa kelimeler eşit değildir. Bu durum derlenen kodda farklı uzunluktaki bir kelimeyle sınandı.

10. Hazır Fonksiyon Ne Zaman Kullanılır?

Araştırma görevlisi hazır fonksiyonlar için bir ayrım yaptı. `malloc`, `calloc` ve `printf` gibi fonksiyonlar kullanılır. Bunların arkasında çok sayıda sistem çağrısı ve mekanizma vardır ve öğrenciden yazması beklenmez. Ama iki stringi karşılaştırmak ya da bir stringin alt stringini (substring) almak gibi basit işler kendi kodunuzla yapılmalıdır. Her string bir karakter dizisidir. String işlemi de aslında dizi işlemidir ve dizi işlemleri bu derste zaten öğretilmektedir.

> **Hocanın vurgusu:** Mecbur kalmadıkça hazır fonksiyon kullanmayın. Basit string ve dizi işlemlerini kendiniz yazın.

Dersin Toparlaması

Bu oturumda iki sorunun çözümü üzerinden aynı fikir işlendi: Doğru veri yapısını seçmek. Renk histogramında bütün pikselleri gezmek kaçınılmazdır, ama farklı renkler dizisindeki arama renk bulununca kesilebilir. Bellek tarafında sabit dizi, `realloc` ve bağlı liste karşılaştırıldı. `realloc` bellek kazandırır ama CPU ve zaman harcatır. Bağlı liste esnektir, ama ek adres tutar ve rastgele erişim sunmaz. Kodlanmış kelime sorusunda ise okuma ve silme yönlerinin farkı, sorunun bir yığın sorusu olduğunu gösterdi. Çözme de karşılaştırma da push ve pop ile yapılabilir.

Bunların yanında iyi kod alışkanlıkları öne çıktı: Dinamik bellek ayırmak, işi bitince `free` çağırmak, anlamlı değişken adları kullanmak, global değişkenden kaçınmak, basit işler için hazır fonksiyona yaslanmamak. Global değişkenlerin yol açtığı yarış durumu, kritik bölge ve mutex konuları işletim sistemleri dersinde ayrıntılı görülecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.